

/api Module Guide

The `api` folder is the FastAPI backend layer — it exposes the platform's core services/ logic over HTTP so any frontend (Streamlit, a mobile app, etc.) can consume it without touching internal Python classes directly.

`main.py`

What it does: The application entrypoint. Creates the FastAPI app, registers all routers, and runs a startup routine that initializes every service once (LLM, RAG, agents) before the server starts accepting requests.

Why it exists: Every FastAPI project needs one place that assembles the app. Keeping startup logic here (rather than scattered across routers) means services are built exactly once, not on every request — faster and safer.

Backend integration: Imports `dependencies.py` to build shared service instances, and includes all router modules so their endpoints become live routes.

`schemas.py`

What it does: Defines the request/response shape for every endpoint (e.g. what a chat request JSON looks like, what a chat response returns) using Pydantic models.

Why it exists: This is the **contract** between backend and frontend. It gives automatic request validation, clear error messages on bad input, and auto-generated API docs — so a frontend developer knows exactly what to send and expect back without reading backend logic.

Backend integration: Routers use these schemas to validate incoming requests and shape outgoing responses. Internally, these get converted to/from the domain models used by `services/` (e.g. `CustomerQuery`, `ChatbotResponse`).

`dependencies.py`

What it does: Builds one shared instance each of the core services — LLM client, RAG store, PII masker, agents, orchestrator — and hands them to routers via FastAPI's dependency injection.

Why it exists: Services like the LLM connection or vector store are expensive to create. This ensures they're built once per app run and reused across all requests, instead of re-initializing on every API call.

Backend integration: This is the bridge — every router calls into `services/` *through* this file, never by importing services directly.

`routers/chat.py`

What it does: Endpoints for the core conversation flow — send a message (POST /api/v1/chat), get history (GET .../history), clear a session (DELETE .../session).

Why it exists: This is the primary endpoint a frontend chat UI will call. One clean endpoint hides the whole multi-agent pipeline (PII masking → routing → agent → response).

Backend integration: Calls `AgentOrchestratorService.handle_customer_query()` — the same orchestrator used in `main_simulation.py`. No duplicate logic.

routers/ingestion.py

What it does: Endpoints to load data into the knowledge base — policies, product catalogs, or a Twitter-support CSV.

Why it exists: Lets an admin/ops tool (or a setup script) populate the RAG knowledge base over HTTP instead of editing Python scripts directly.

Backend integration: Calls `DataIngestionPipeline` methods, then the data becomes searchable via `RAGService` for all future chat requests.

routers/evaluation.py

What it does: One endpoint (POST /api/v1/evaluate) to run the evaluation framework (retrieval quality, faithfulness, latency, etc.) against a labeled test batch.

Why it exists: Lets QA/testing tooling trigger evaluations remotely rather than running Python scripts locally.

Backend integration: Calls `EvaluationService.build_report()`.

routers/health.py

What it does: GET /api/v1/health — reports whether the LLM/RAG are configured correctly and the knowledge base size.

Why it exists: Standard practice for any deployed API — lets monitoring tools (and your Streamlit app) check if the backend is actually ready before sending real traffic.

Backend integration: Reads live status from the same singletons in `dependencies.py`.

For the Streamlit developer

- Run the backend: `uvicorn api.main:app --reload`
- Explore live docs: `http://localhost:8000/docs` (interactive — test every endpoint from the browser)
- Only `schemas.py` + the endpoint URLs matter for frontend work — no need to read `services/`